

ՆԵՐԱԾՈՒԹՅՈՒՆ

Թեմայի արդիականությունը

Ժամանակակից կիրառական մաթեմատիկայում և համակարգչային գիտություններում գրաֆների տեսությունը հանդիսանում է հիմնարար գործիք բարդ համակարգերի մոդելավորման և օպտիմիզացիայի համար: Համիլտոնյան ցիկլի որոնման խնդիրը (Hamiltonian Cycle Problem - HCP) դիսկրետ մաթեմատիկայի դասական և ամենահայտնի խնդիրներից մեկն է, որը պատկանում է NP-լրիվ (NP-complete) խնդիրների դասին: Սա նշանակում է, որ գազաթների քանակի մեծացմանը զուգընթաց, բոլոր հնարավոր լուծումների որոնման ժամանակն աճում է էքսպոնենցիալ կերպով, ինչը սահմանափակում է ավանդական ալգորիթմների կիրառելիությունը մեծ չափերի գրաֆների համար:

Ուղղանկյուն ցանցային գրաֆների (grid graphs) ուսումնասիրությունը հատուկ արդիականություն ունի, քանի որ այն ուղղակիորեն կիրառվում է գործնական բազմաթիվ ոլորտներում՝ սկսած ռոբոտատեխնիկայից (տարածության օպտիմալ շրջանցում) և ինտեգրալ միկրոսխեմաների նախագծումից (VLSI design), մինչև տվյալների կոդավորում և բարդ մոլեկուլային կառույցների մոդելավորում: Չնայած այն հանգամանքին, որ հատուկ դասի գրաֆների համար գոյություն ունեն որոշակի էվրիստիկ մոտեցումներ, արդյունավետ և ճշգրիտ հաշվողական ծրագրային համալիրների մշակումը շարունակում է մնալ գիտական լուրջ մարտահրավեր:

Հետազոտության նպատակը և խնդիրները

Աշխատանքի հիմնական նպատակն է $m \times n$ չափանի ուղղանկյուն ցանցային գրաֆներում Համիլտոնյան ցիկլերի որոնման և ճշգրիտ հաշվարկման համար մշակել, օպտիմիզացնել և ծրագրավորել բարձր արդյունավետություն ունեցող ալգորիթմներ:

Այս նպատակին հասնելու համար առաջադրվել և լուծվել են հետևյալ խնդիրները.

1. Ուսումնասիրել գրաֆների տեսության և Համիլտոնյան ցիկլերին վերաբերող մաթեմատիկական հիմքերն ու տեսությունները:
2. Վերլուծել և փորձարկել խնդրի լուծման այլընտրանքային ճարտարապետություններ (GPU զուգահեռացում, մաքուր ռեկուրսիա, բարդ բեքթրեքինգ (backtracking))՝ բացահայտելով դրանց սահմանափակումները:
3. Խորության որոնման (DFS) ալգորիթի հիման վրա մշակել ճշգրիտ հաշվարկի օպտիմիզացված ծրագրային լուծում՝ կիրառելով բիթային դիմակներ (bitmasks) և JIT-կոմպիլյացիա:
4. Մշակել մեծ չափերի գրաֆների համար էվրիստիկ ալգորիթ՝ կիրառելով Վառնսդորֆի կանոնը (Warnsdorff's rule) և ադապտիվ պատահականացում:
5. Իրականացնել ստացված տվյալների վերլուծություն և լուծումների վիզուալիզացիա:

Հետազոտության օբյեկտը և առարկան

Հետազոտության օբյեկտ են հանդիսանում վերջավոր, կապակցված, ուղղված և չուղղված ցանցային գրաֆները:

Հետազոտության առարկան այդ գրաֆներում Համիլտոնյան ցիկլերի ճշգրիտ հաշվարկման և արագ որոնման ալգորիթմական ու ծրագրային մեթոդներն են:

ԳԼՈՒԽ 1. ՀԵՏԱԶՈՏՈՒԹՅԱՆ ՏԵՍԱԿԱՆ ՀԻՄՔԵՐԸ

1.1. Գրաֆների տեսության հիմնական հասկացությունները

Գրաֆը մաթեմատիկական կառույց է, որը նկարագրվում է որպես $G = (V, E)$, որտեղ V -ն գագաթների (հանգույցների) ոչ դատարկ բազմությունն է, իսկ E -ն՝ կողերի բազմությունը, որոնք միացնում են գագաթների զույգերը: Եթե կողերը ուղղություն չունեն, գրաֆը կոչվում է չուղղված:

Ցանցային գրաֆ (Grid graph): Ուղղանկյուն ցանցային գրաֆը, որը նշանակվում է $L_{m,n}$, ձևավորվում է երկու ուղիների գրաֆների դեկարտյան արտադրյալից՝ $P_m \square P_n$: $L_{m,n}$ գրաֆի գագաթները կարող են ներկայացվել որպես կոորդինատային զույգեր (i, j) , որտեղ $1 \leq i \leq m$ և $1 \leq j \leq n$: Երկու գագաթ՝ (i_1, j_1) և (i_2, j_2) , միացված են կողով այն և միայն այն դեպքում, երբ $|i_1 - i_2| + |j_1 - j_2| = 1$: Այսպիսով, գրաֆի յուրաքանչյուր ներքին գագաթ ունի ճիշտ 4 հարևան (աստիճանը հավասար է 4), եզրային գագաթները՝ 3, իսկ անկյունային գագաթները՝ 2:

Ուղի և Ցիկլ: Գրաֆում $v_1, v_2, \dots, v_k$ գագաթների հաջորդականությունը կոչվում է *ուղի*, եթե յուրաքանչյուր հարևան զույգ $(v_i, v_{i+1}) \in E$: Ուղին կոչվում է *պարզ*, եթե բոլոր գագաթները եզակի են: *Ցիկլը* փակ պարզ ուղի է, որտեղ հաջորդականության առաջին և վերջին գագաթները համընկնում են ($v_1 = v_k$), իսկ մնացած բոլոր գագաթները եզակի են:

1.2. Համիլտոնյան գրաֆներ և թեորեմներ

Համիլտոնյան ցիկլի սահմանումը: Համիլտոնյան ցիկլը G գրաֆում այնպիսի պարզ ցիկլ է, որն անցնում է գրաֆի յուրաքանչյուր գագաթով ճիշտ մեկ անգամ: Գրաֆը, որը պարունակում է առնվազն մեկ Համիլտոնյան ցիկլ, կոչվում է *Համիլտոնյան գրաֆ*:

Ի տարբերություն Էյլերյան ցիկլերի, որոնց գոյության պայմանները ձևակերպվում են պարզ թեորեմներով, գրաֆի Համիլտոնյան լինելու ընդհանուր անհրաժեշտ և բավարար պայման գոյություն չունի: Սակայն ցանցային գրաֆների $L_{m,n}$ դեպքում կարելի է հստակ մաթեմատիկական պայմաններ սահմանել:

Թեորեմ (Ցանցային գրաֆում Համիլտոնյան ցիկլի գոյության պայմանը):

$L_{m,n}$ ուղղանկյուն ցանցային գրաֆը Համիլտոնյան է այն և միայն այն դեպքում, եթե $m \cdot n$ արտադրյալը զույգ թիվ է, և $m, n > 1$:

Ապացույց (Անհնարինությունը կենտ քանակով գագաթներով գրաֆներում):

Ապացույցը հիմնված է այն փաստի վրա, որ ցանկացած $L_{m,n}$ ցանցային գրաֆ *երկկողմանի գրաֆ է (bipartite graph)*:

Ենթադրենք գրաֆի գագաթների բազմությունը V է, որտեղ $|V| = m \cdot n$: Մենք կարող ենք ներկել գրաֆի գագաթները շախմատային տախտակի սկզբունքով (սև և սպիտակ), այնպես որ ցանկացած կողմից միացնում է տարբեր գույնի գագաթներ: Դիցուք՝ V_1 -ը սև գագաթների բազմությունն է, իսկ V_2 -ը՝ սպիտակ:

Ակնհայտ է, որ ցանկացած ցիկլ, որն անցնում է երկկողմանի գրաֆով, պետք է խստորեն հերթագայի գույները՝ սև $\rightarrow$ սպիտակ $\rightarrow$ սև $\rightarrow$ սպիտակ...:

Հետևաբար, գրաֆի բոլոր գագաթներն ընդգրկող ցիկլի գոյության համար անհրաժեշտ է, որ գույների քանակը հավասար լինի՝ $|V_1| = |V_2|$:

Եթե $m \cdot n$ -ը կենտ է, ապա գրաֆի ընդհանուր գագաթների քանակը կենտ է: Այս դեպքում անհնար է բազմությունը կիսել երկու հավասար մասի ($|V_1| \neq |V_2|$): Քանի որ գագաթների քանակն անհավասար է, կատարյալ հերթագայությամբ փակ ցիկլ կառուցելն անհնար է: Թեորեմն ապացուցված է: Այս մաթեմատիկական կանոնը որպես հիմնարար ֆիլտր ներդրվել է մեր մշակած էվրիստիկ ալգորիթմներում (parity check):

1.3. Գոյություն ունեցող լուծման մեթոդների ակնարկ

Համիլտոնյան ցիկլի գտման կամ հաշվման համար ժամանակակից գրականության մեջ կիրառվում են մի շարք մոտեցումներ, որոնք ունեն իրենց առավելություններն ու թերությունները.

1. Ամբողջական ուժեղ (Brute Force / Backtracking):

Հիմնված է խորության որոնման (DFS) վրա: Մեթոդը ստուգում է բոլոր հնարավոր ուղիները և չեղարկում է (backtrack) այն ճյուղերը, որոնք փակուղի են: Ճշգրիտ է, սակայն ժամանակային բարդությունը կազմում է $O(N!)$ վատագույն դեպքում:

2. Դինամիկ ծրագրավորում (Dynamic Programming - DP):

Ցանցային գրաֆների համար հաճախ կիրառվում է Transfer Matrix Method (Փոխանցման մատրիցների մեթոդ) կամ վիճակների պրոֆիլավորում (Profile DP): Այս մեթոդները զգալիորեն արագացնում են հաշվարկը որոշակի հաստատուն լայնություն ունեցող ցանցերի (օրինակ՝ $m \times n$, որտեղ $m \leq 10$) դեպքում, սակայն պահանջում են հսկայական ծավալի օպերատիվ հիշողություն՝ բոլոր հնարավոր վիճակները պահելու համար:

3. Էվրիստիկ և մոտարկման ալգորիթմներ:

Մեծ գրաֆների համար, որտեղ ճշգրիտ մեթոդները անգոր են, կիրառվում են գենետիկ ալգորիթմներ, մրջյունների գաղութի ալգորիթմներ (Ant Colony Optimization) կամ տեղային օպտիմիզացիայի կանոններ, ինչպիսին է Վառնսդորֆի կանոնը (որն ի սկզբանե նախագծված է եղել շախմատի ձիու պտույտի համար, բայց տրամաբանությունը կիրառելի է ընդհանուր գրաֆների համար):

4. Զուգահեռ հաշվարկներ (GPU / Մուլտիպրոցեսորային հաշվարկներ):

Տեսականորեն, որոնման ծառի տարբեր ճյուղեր կարելի է զուգահեռացնել: Սակայն, ինչպես ցույց կտան մեր հետազոտության արդյունքները և էվոլյուցիան (մանրամասն քննարկված Գլուխ 2-ում), գրաֆների որոնման ասիմետրիկ բնույթը GPU ճարտարապետության վրա առաջացնում է հոսքերի տարամիտման (thread divergence) լուրջ խնդիրներ:

ԳԼՈՒԽ 2. ԾՐԱԳՐԱՅԻՆ ԻՐԱԿԱՆԱՑՈՒՄ, ՄՈՏԵՑՈՒՄՆԵՐԻ ԷՎՈԼՅՈՒՑԻԱ ԵՎ ԱԼԳՈՐԻԹՄՆԵՐԻ ՕՊՏԻՄԻԶԱՅԻԱ

2.1. Տեխնոլոգիական գործիքակազմի ընտրության հիմնավորում

Աշխատանքում առաջադրված մաթեմատիկական և ալգորիթմական բարդ խնդիրների լուծման համար որպես հիմնական ծրագրավորման լեզու ընտրվել է Python-ը: Չնայած

այն հանգամանքին, որ C++ կամ Rust լեզուներն ապահովում են կատարման ավելի բարձր արագություն բազային մակարդակում, Python-ն ընտրվել է իր բարձր ճկունության, ծրագրային ճարտարապետության արագ նախատիպավորման (prototyping) և հզոր գրադարանային էկոհամակարգի պատճառով:

Կատարողականության խնդիրը, որը բնորոշ է ինտերպրետացվող լեզուներին, հաջողությամբ լուծվել է **Numba** գրադարանի ներդրմամբ: Numba-ն հնարավորություն է ընձեռում կատարել JIT (Just-In-Time) կոմպիլյացիա, ինչը ապահովում է Python կոդի կատարման արագություն՝ համեմատելի կոմպիլյացվող լեզուների հետ (օրինակ՝ C/C++): Ալգորիթմի արդյունքների վերլուծության, ինչպես նաև որոնման գործընթացի անիմացիոն վիզուալիզացիայի համար ընտրվել է **Matplotlib** (ներառյալ numpy և matplotlib.patches մոդուլները) գրադարանը:

2.2. Միջանկյալ և անհաջող մոտեցումների վերլուծություն

Մինչև վերջնական, բարձր արդյունավետություն ունեցող ծրագրային համալիրների կառուցումը, կատարվել է տարբեր ճարտարապետական լուծումների ծավալուն հետազոտություն: Ստորև ներկայացված է այդ մոտեցումների էվոլյուցիան և դրանց ձախողման տեխնիկական պատճառները.

1. Մաքուր ռեկուրսիա Python միջավայրում (առանց Numba-ի և բիթային դիմակների):

Առաջին փորձնական տարբերակները հիմնված էին ստանդարտ ռեկուրսիվ որոնման վրա, որտեղ վիճակները պահվում էին ցուցակների (list) կամ բազմությունների (set) տեսքով: Սակայն, Python ինտերպրետատորի լրացուցիչ ծանրաբեռնվածության (overhead) և հիշողության դինամիկ տեղաբաշխման պատճառով արագությունը խիստ ցածր էր, և ալգորիթմն ի վիճակի չէր հաշվարկել նույնիսկ 6×6 չափի գրաֆի ցիկլերը ողջամիտ ժամանակահատվածում:

2. Զուգահեռացման փորձեր գրաֆիկական պրոցեսորների (GPU) վրա (CUDA / PyCUDA):

Ալգորիթմական արագացման նպատակով առաջադրվեց որոնման ծառի ճյուղերի զուգահեռացման գաղափարը վիդեոքարտի միջոցով: Չնայած այն բանին, որ GPU-ն հզոր է մատրիցային և միատեսակ հաշվարկներում, գրաֆներում խորության որոնման (DFS) առանձնահատկությունները հանգեցրին երկու լուրջ տեխնիկական խնդրի.

- *Հոսքերի տարամիտում (Thread Divergence):* Քանի որ յուրաքանչյուր հոսք (thread) գնում է որոնման ծառի տարբեր ճյուղերով, պրոցեսորային հրահանգների կատարման սիմետրիան խախտվում է, ինչը հանգեցնում է հոսքերի պարապուրդի:
- *Լոկալ հիշողության գերբեռնում (Local Memory Overflow):* Յուրաքանչյուր հոսք պետք է պահեր իր յուրահատուկ վիճակը (այցելված գագաթների պատմությունը), ինչը արագորեն սպառեց GPU-ի լոկալ հիշողությունը վիճակների գրաֆի խորը բազմացման պատճառով:

3. Բարդ բեքթրեքինգի (Backtracking) ռազմավարություններ:

Միջանկյալ տարբերակներում կիրառվել են հիշողության մեջ վիճակների պահպանման բարդ համակարգեր (DeepBacktrack, TimeCheckpoints):

Այնուամենայնիվ, այս կառույցների պահպանման ու թարմացման վրա ծախսվող ժամանակային և հիշողության ռեսուրսներն ավելին էին, քան բուն հաշվարկինը: Պարզվեց, որ պարզ և թեթևագույն բիթային գործողությունները ապահովում են շատ ավելի մեծ արագություն:

Այս վերլուծությունը թույլ տվեց հիմնավորել անցումը դեպի օպտիմիզացված պրոցեսորային (CPU) հաշվարկներ՝ կիրառելով բիթային դիմակներ և Numba:

2.3. Համիլտոնյան ցիկլերի ճշգրիտ հաշվարկման ալգորիթմ (Calculate_alg_V3.0.1.0.py)

Վերջնական արդյունքում մշակված Calculate_alg_V3.0.1.0.py ծրագրային մոդուլի նպատակն է գտնել նշված չափերի ցանցային գրաֆի բոլոր հնարավոր Համիլտոնյան ցիկլերի հստակ քանակը: Ալգորիթմի հիմքում ընկած է խորության որոնման (DFS) մեթոդը, որն օժտված է հետևյալ ճարտարապետական օպտիմիզացիաներով.

- **ՅՈՒ Կոմպիլյացիա:** Խիստ հաշվողական ֆունկցիաները ձևավորվել են Numba-ի @jit(cache=True) դեկորատորով, որը կոմպիլացնում է Python կոդը օպտիմիզացված մեքենայական կոդի՝ բազմապատկելով կատարման արագությունը: Քեշավորման (cache=True) ռեժիմը թույլ է տալիս խուսափել կրկնակի կոմպիլյացիայից հաջորդ գործարկումների ժամանակ:
- **Բիթային դիմակներ (Bitmasks):** Որպես հիշողության օպտիմիզացիայի գլխավոր լուծում՝ այցելված գագաթների բազմությունը պահվում է ոչ թե զանգվածների, այլ մեկ 64-բիթանոց կամ ավելի մեծ ամբողջ թվի մեջ: Օրինակ՝ 10×6 գրաֆի դեպքում (60 գագաթ) պահանջվում է ընդամենը 60 բիթ տեղեկատվություն վիճակը հիշելու համար: Սա ոչ միայն խնայում է հիշողությունը, այլև թույլ է տալիս գագաթի ստուգումը և գրանցումը կատարել $O(1)$ բարդությամբ՝ հիմնվելով արագ բիթային գործողությունների վրա (Bitwise AND, OR, XOR):
- **Ալգորիթմական էտում (Pruning):** Ներդրվել է հատուկ մշակված is_connected ֆունկցիան, որը խիստ կարևոր դեր ունի փակուղային ճյուղերի (dead ends) վաղաժամ հայտնաբերման գործում: Այն ստուգում է արդյոք գրաֆի դեռևս չայցելված մասը ժամում է կապակցված, թե ոչ: Եթե ընթացիկ քայլի արդյունքում չայցելված գագաթները բաժանվում են երկու կամ ավելի անջատ բաղադրիչների, պարզ է դառնում, որ Համիլտոնյան ցիկլ կառուցելն արդեն անհնար է, և ալգորիթմը անմիջապես վերադառնում է նախորդ վիճակին (backtrack)՝ բացառելով միլիոնավոր ավելորդ քայլեր:

- **Ավտոմատացված անալիտիկա:** Մշակված է ավտոմատացման սկրիպտ, որն ինքնուրույն վերլուծում է Markdown ֆորմատով արդյունքների աղյուսակը, հայտնաբերում դեռևս չհաշվարկված $m \times n$ չափերը, հաշվարկում է դրանք և ավտոմատ թարմացնում վերջնական ֆայլը՝ գրանցելով նաև կատարման ժամանակագրությունը:

2.4. Խոշոր գրաֆների համար էվրիստիկ ալգորիթմի մշակում ([HCFA.py](#))

Այն դեպքերում, երբ գրաֆի չափերը գերազանցում են ճշգրիտ հաշվարկման հնարավորությունները (խնդրի NP-լրիվ բնույթի պատճառով), նպատակ է դրվել գտնել գոնե մեկ Համիլտոնյան ցիկլ: Այս նպատակով ստեղծվել է նպատակային էվրիստիկ ալգորիթմ ([HCFA.py](#)).

- **Վառնսդորֆի կանոնի (Warnsdorff's Rule) և պատահականացման հիբրիդավորում:** Ալգորիթմի հիմքում ընկած է հայտնի Վառնսդորֆի կանոնը, ըստ որի՝ հաջորդ քայլի համար ընտրվում է այն հարևան գագաթը, որն ունի ամենաքիչ քանակությամբ ազատ (չայցելված) հարևաններ: Քանի որ մաքուր էվրիստիկան կարող է ցանցային գրաֆներում արագ մտնել փակուղի, մեր լուծման մեջ ինտեգրվել է պատահականության կշիռ (random weight):
- **Ադապտիվ վերագործարկում (Adaptive Restarts):** Եթե ալգորիթմը մտնում է տեղային մինիմումների մեջ (local minima) և չի կարողանում գտնել լուծում, այն կատարում է վերագործարկում (restart)՝ աստիճանաբար մեծացնելով պատահականության գործակիցը, ինչը թույլ է տալիս դուրս գալ կրկնվող անարդյունավետ սցենարներից:
- **Նախնական ֆիլտրացիա:** Ինչպես հիմնավորվեց Գլուխ 1-ում, ալգորիթմը կատարում է զույգության ստուգում (parity check). Եթե $m \times n$ արտադրյալը կենտ է, ծրագիրը անմիջապես դադարեցնում է աշխատանքը՝ առանց հաշվողական ռեսուրսներ վատնելու, քանի որ լուծումն անհնար է մաթեմատիկորեն:

- **Վիզուալիզացիոն և անիմացիոն մոդուլ:** Matplotlib գրադարանի բազայի վրա նախագծվել է գործիքակազմ, որը ոչ միայն գրաֆիկորեն ներկայացնում է հայտնաբերված լուծումը, այլև կառուցում է «ջերմային քարտեզ» (heatmap), որն արտացոլում է գրաֆի շրջանցման կարգն ու դինամիկան: Սա թույլ է տալիս վիզուալ գնահատել էվրիստիկ ալգորիթմի վարքագիծը և մեկուսացնել խնդրահարույց գոտիները գրաֆի եզրերում և անկյուններում:

ԳԼՈՒԽ 3. ՓՈՐՁԱՐԱՐԱԿԱՆ ԱՐԴՅՈՒՆՔՆԵՐԻ ՎԵՐԼՈՒԾՈՒԹՅՈՒՆ ԵՎ ՎԻԶՈՒԱԼԻԶԱՑԻԱ

3.1. Հաշվարկային աղյուսակի վերլուծություն և ալգորիթմի արդյունավետության գնահատում

Մշակված Calculate_alg_V3.0.1.0.py ծրագրային ապահովման միջոցով իրականացվել են բազմաթիվ հաշվարկներ՝ տարբեր չափերի ցանցային գրաֆների համար:

Ստացված տվյալները հավաքագրվել և ավտոմատ կերպով արձանագրվել են վերջնական արդյունքների աղյուսակում (Calculate Table [V3.0.1.0.md](#)):

Փորձարկումների ընթացքում ճշգրիտ հաշվարկման ալգորիթմը ցուցաբերել է բարձր կայունություն և հաջողությամբ գտել է Համիլտոնյան ցիկլերի ճշգրիտ քանակները ընդհուպ մինչև 10x6 չափանի գրաֆների համար: Մասնավորապես, 10x6 չափանի ցանցում հայտնաբերվել է ճիշտ 1,024,028 եզակի Համիլտոնյան ցիկլ: Numba գրադարանի և բիթային դիմակների ընձեռած օպտիմիզացիաների շնորհիվ այս ծավալի բարդագույն հաշվարկը պահանջել է մոտավորապես 2 րոպե կատարման ժամանակ, ինչը զգալի առաջընթաց է NP-լրիվ խնդիրների դասական լուծումների համեմատությամբ:

Արդյունքների մաթեմատիկական և վիճակագրական վերլուծությունը բացահայտում է մի քանի առանցքային օրինաչափություններ.

- **Միմետրիայի (Համաչափության) հատկություն:** Աղյուսակային տվյալները գործնականում հաստատում են մաթեմատիկական ակնկալիքը, առ այն, որ $m \times n$ չափի գրաֆում ցիկլերի քանակը խստորեն հավասար է $n \times m$ չափի գրաֆի ցիկլերի քանակին (գրաֆի 90 աստիճանով պտտման հատկություն):
- **Նեղ և կենտ ցանցերի սահմանափակումներ:** Արդյունքները ցույց են տալիս, որ նեղ ցանցերում ցիկլերի ձևավորումը խիստ սահմանափակ է: $1 \times N$ չափանի ցանցերի դեպքում անհնար է փակ ցիկլ կառուցել՝ առանց նույն գագաթով կրկին անցնելու: Ինչպես նաև, կենտ քանակով գագաթներ պարունակող գրաֆների համար ալգորիթմը, տեսական ապացույցի համաձայն, անմիջապես վերադարձնում է 0 արդյունք:
- **Ժամանակային բարդության էքսպոնենցիալ աճ:** Գրաֆի չափերի (գագաթների քանակի) մեծացմանը զուգընթաց ակնհայտորեն նկատվում է ալգորիթմի կատարման ժամանակի և հայտնաբերված ցիկլերի քանակի էքսպոնենցիալ աճ: Այս փաստը ևս մեկ անգամ վկայում է Համիլտոնյան ցիկլի որոնման խնդրի հաշվողական բարդության մասին և սահմանում է այն տրամաբանական շեմը, որից այն կողմ ճշգրիտ ամբողջական որոնումը (brute force) դառնում է աննպատակահարմար՝ անհրաժեշտություն ստեղծելով կիրառել էվրիստիկ մոտեցումներ:

3.2. Արդյունքների վիզուալիզացիա և անիմացիա

Էվրիստիկ ալգորիթմի ([HCFAPy](https://github.com/HCFAPy)) աշխատանքի տրամաբանությունը հասկանալու և արդյունքները մեկնաբանելու նպատակով մշակվել է վիզուալիզացիայի հատուկ մոդուլ՝ հիմնված Matplotlib գրադարանի վրա: Այն ոչ միայն ստատիկ կերպով արտապատկերում է գտնված ուղին, այլև ապահովում է խորքային վերլուծական գործիքակազմ.

- **Ջերմային քարտեզի (Heatmap) կառուցում:** Մոդուլը գեներացնում է գրաֆի շրջանցման ջերմային քարտեզ, որտեղ գագաթների գունային երանգավորումը ցույց է տալիս ցիկլի կառուցման ժամանակագրական հաջորդականությունը: Սա հնարավորություն է ընձեռում հետազոտել, թե որոնման ընթացքում ալգորիթմը որ գոտիներն է ավելի վաղ սպասարկում, և որտեղ են կուտակվում խնդրահարույց չայցելված հանգույցները:
- **Քայլ առ քայլ անխմացիա:** Անխմացիոն գործիքակազմի միջոցով ծրագիրը ցուցադրում է Համիլտոնյան ցիկլի կառուցման դինամիկ գործընթացը հարաբերական իրական ժամանակում: Այս գործառնությունը խիստ արժեքավոր է վարքագծային անոմալիաները (օրինակ՝ անընդհատ միևնույն փակուղին մտնելը) հայտնաբերելու և էվրիստիկ կանոնների (մասնավորապես՝ վառնսդորֆյան և պատահական քաշերի հարաբերակցության) ճշգրտման համար:

3.3. $m \times n$ և $m \times (n - 1)$ չափանի ցանցային գրաֆներում Համիլտոնյան ցիկլերի քանակական օրինաչափությունների և ռեկուրենտ կապերի վերլուծություն

Հետազոտության փորձարարական փուլում հաշվարկային աղյուսակի տվյալների համալիր վերլուծությունը բացահայտեց կարևոր անալիտիկ օրինաչափություն, որը կապում է հարևան չափողականություն ունեցող գրաֆների արդյունքները:

Մասնավորապես, խնդիր դրվեց ուսումնասիրել, թե ինչպես է գրաֆի միակողմանի ընդլայնումը (մեկ սյան կամ տողի ավելացումը) ազդում Համիլտոնյան ցիկլերի ընդհանուր քանակի վրա, և արդյոք հնարավոր է մոդելավորել այդ անցումը մաթեմատիկորեն:

Դիցուք $H(m, n)$ -ով նշանակված է $m \times n$ չափանի ցանցային գրաֆում առկա Համիլտոնյան ցիկլերի ճշգրիտ քանակը: Տրամաբանական է դիտարկել $H(m, n)$ և $H(m, n - 1)$ մեծությունների միջև գոյություն ունեցող ֆունկցիոնալ

և ռեկուրենտ կախվածությունը՝ պարզելու համար $H(m, n) = f(H(m, n - 1))$ հարաբերակցության բնույթը:

Ընդլայնման տոպոլոգիական մեխանիզմը

Երբ $m \times (n - 1)$ չափանի գրաֆին ավելացվում է լրացուցիչ սյուն՝ բաղկացած m գագաթներից, նոր Համիլտոնյան ցիկլերի գեներացիան հիմնվում է նախկին ցանցում հնարավոր ուղիների «տրանսֆորմացիայի» վրա: Տեսական տեսանկյունից այս գործընթացը կարելի է նկարագրել որպես գոյություն ունեցող փակ ցիկլերի նպատակային խզում (շրջանցման որոշակի կողերի հեռացում) և նոր m գագաթների ինտեգրում այդ խզման հատվածներում՝ պահպանելով գրաֆի Համիլտոնյան բնույթը: Այս մեխանիզմը ենթադրում է, որ $H(m, n)$ -ի արժեքը խիստ պայմանավորված է ոչ միայն $H(m, n - 1)$ -ի բացարձակ արժեքով, այլև նախորդ ցանցի եզրային կողերի կոնֆիգուրացիայով:

Աճի դինամիկան և մաթեմատիկական հարաբերակցությունը

Էմպիրիկ տվյալների նախնական ուսումնասիրությունը թույլ է տալիս արձանագրել, որ ֆիքսված m -ի դեպքում n -ի աճին զուգընթաց $H(m, n)$ ֆունկցիան ցուցաբերում է էքսպոնենցիալ աճի միտում: Կախվածությունը կարող է մոդելավորվել հետևյալ ընդհանրացված մոտարկմամբ.

$$H(m, n) \approx \alpha_m \cdot H(m, n - 1)$$

որտեղ α_m -ը աճի սպեցիֆիկ գործակիցն է (կամ բազմապատկեր), որը կախված է գրաֆի ֆիքսված լայնությունից (m): Ավելի խորը հանրահաշվական մոտեցման դեպքում (օրինակ՝ անցումային մատրիցների մեթոդի՝ Transfer Matrix Method-ի տեսանկյունից), յուրաքանչյուր ֆիքսված m -ի համար հնարավոր է ստանալ գծային ռեկուրենտ հավասարում հաստատուն գործակիցներով:

Քանի որ $H(m, n)$ ֆունկցիան n -ի աճին զուգընթաց դրսևորում է էքսպոնենցիալ վարքագիծ, հատուկ հետաքրքրություն է ներկայացնում աճի գործակցի ասիմպտոտիկ սահմանի գնահատումը: Դիցուք y_n -ով նշանակված է աճի հարաբերակցությունը յուրաքանչյուր n -րդ քայլում՝ ֆիքսված m -ի դեպքում (այսինքն՝ $y_n = \frac{H(m, n)}{H(m, n-1)}$): Որպեսզի գնահատենք այս հաջորդականության իրական սահմանը ($L = \lim_{n \rightarrow \infty} y_n$) սահմանափակ փորձարարական տվյալների պայմաններում, կիրառվել է զուգամիտության արագացման մաթեմատիկական մոտեցում:

Աճի գործակցի ճշգրտված սահմանը հաշվարկվում է հետևյալ մոտարկման բանաձևով, որն օգտագործում է վերջավոր տարբերությունները՝

$$L \approx y_n - \frac{(y_n - y_{n-1})^2}{(y_n - y_{n-1}) - (y_{n-1} - y_{n-2})}$$

որտեղ y_n , y_{n-1} և y_{n-2} արժեքները աճի գործակցի իրար հաջորդող հայտնի արդյունքներն են: Այս բանաձևի կիրառումը (հայտնի որպես Այտկենի պրոցես) թույլ է տալիս արդյունավետորեն հարթել հաջորդականության անցումային տատանումները և ստանալ α_m աճի գործակցի շատ ավելի ճշգրիտ ասիմպտոտիկ արժեք՝ հիմնվելով ճշգրիտ հաշվարկման ալգորիթմի գեներացրած նախորդող երեք քայլերի տվյալների վրա: Ստացված սահմանային արժեքը (L) էականորեն մեծացնում է ավելի մեծ ցանցերի (որտեղ $n \gg 1$) համար արվող անալիտիկ կանխատեսումների վստահելիությունը և տալիս է տեսական հիմնավորում էվրիստիկ ալգորիթմների ակնկալվող բարդության գնահատման համար:

Ուղղանկյուն ցանցային գրաֆներում կոմբինատորային օպտիմիզացիայի խնդիրներում (մասնավորապես՝ Համիլտոնյան ցիկլերի որոնման և հաշվարկման ժամանակ) էքսպոնենցիալ կախվածության ի հայտ գալը միանգամայն օրինաչափ երևույթ է: Նման կառույցների համար հնարավոր կոնֆիգուրացիաների կամ ուղիների քանակը հաճախ աճում է $\lambda^{m \cdot n}$ -ին համեմատական, որտեղ λ -ն

հաստատուն է՝ կապված այսպես կոչված անցումային մատրիցի (transfer matrix) գերիշխող սեփական արժեքի (dominant eigenvalue) հետ:

Փոքր ցանցերում եզրային պայմանների (boundary conditions) պատճառով $m \times n$ և $m \times (n - 1)$ չափերի երթուղիների քանակի հարաբերությունը կարող է անմիջապես ասիմպտոտիկ հաստատուն արժեքի չհասնել, այլ դրսևորել սահուն աճ կամ նվազում, որը բարձր ճշտությամբ մոտարկվում է էքսպոնենցիալ ֆունկցիայով: Զանի որ էքսպոնենցիալ կախվածությունն արդեն իսկ հիմնավորված է, անհայտ արժեքների (օրինակ՝ հաջորդող հարաբերակցության) կանխատեսման ամենաճշգրիտ և մաթեմատիկորեն խիստ մեթոդը տվյալների գծայնացումն է (linearization)՝ լոգարիթմական միջոցով: Այս մոտեցումը գործնականում շատ ավելի հուսալի է, քան բազմանդամային ինտերպոլյացիան, քանի որ էականորեն մեղմում է գրաֆիկի ծայրերում հնարավոր կտրուկ շեղումների (outliers) ազդեցությունը:

Կանխատեսման մաթեմատիկական ալգորիթմը

Աճի դինամիկան կանխատեսելու համար ծրագրային և անալիտիկ մակարդակում կիրառվել է հաշվարկների հետևյալ հաջորդականությունը.

1. Անցում լոգարիթմական տարածություն

Դիցուք, ուսումնասիրվող հարաբերությունները նկարագրվում են հետևյալ ֆունկցիայով.

$$R_n = A \cdot e^{B \cdot n}$$

Հաշվելով յուրաքանչյուր արժեքի բնական լոգարիթմը՝ հավասարումը բերվում է գծային տեսքի.

$$\ln(R_n) = B \cdot n + \ln(A)$$

2. Փոքրագույն քառակուսիների մեթոդի (ՓՔՄ / OLS) կիրառում

Այս ձևափոխության արդյունքում էմպիրիկ արժեքները վերածվում են $(n, \ln(R_n))$ կոորդինատներով կետերի, որոնք գրաֆիկի վրա տեսականորեն

պետք է դասավորվեն ուղիղ գծով: Խնդիրը հանգում է նրանց միջով անցնող օպտիմալ $y = B \cdot x + C$ ուղղի կառուցմանը: Ծրագրային ապահովման շրջանակներում, Python միջավայրում սա արդյունավետ իրագործվել է numpy գրադարանի օգնությամբ.

Python

```
import numpy as np
```

```
# n_values - ինդեքսների զանգված (օրինակ՝ սկսած 1-ից)
```

```
# log_R_values - լոգարիթմված հարաբերությունների զանգված
```

```
B, C = np.polyfit(n_values, log_R_values, 1)
```

```
# Վերադարձ սկզբնական գործակիցներին
```

```
A = np.exp(C)
```

Այստեղ polyfit ֆունկցիան՝ տրված 1 պարամետրով (առաջին աստիճանի բազմանդամ), հաշվարկում է ուղղի օպտիմալ գործակիցները՝ հարթեցնելով այն միկրոշեղումները, որոնք առաջանում են ցանցի եզրային պայմանների խախտումներից:

3. Արդյունքների Էքստրապոլյացիա

Ստանալով A և B գործակիցների հստակ արժեքները՝ հնարավոր է հաշվարկել գրաֆիկի հաջորդ, չհետագոտված տարրը (օրինակ՝ տեղադրելով անհրաժեշտ անհայտ n -ը սկզբնական բանաձևի մեջ).

$$R_{n+1} = A \cdot e^{B \cdot (n+1)}$$

Այս մոտեցումը ապահովում է նոր տարրի առավելագույն ճշգրիտ կանխատեսում՝ հիմնվելով ամբողջ ընտրանքի ընդհանուր վիճակագրական միտումի (trend), այլ ոչ թե միայն վերջին մի քանի արժեքների վրա: Նույնիսկ եթե փոքր չափերի ցանցերում նկատվում են որոշակի տոպոլոգիական անոմալիաներ, ընդհանուր լոգարիթմական հարթեցումը կոմպենսացնում է այդ շեղումները՝ տրամադրելով կայուն էքստրապոլյացիոն մոդել:

Գործնական նշանակությունը և կիրառելիությունը

Նման օրինաչափությունների ճշգրիտ հայտնաբերումն ու ձևակերպումը ունի հիմնարար ալգորիթմական նշանակություն: Եթե հնարավոր է սահմանել խիստ ռեկուրենտ կապ, ապա այն թույլ կտա դուրս գալ խորության որոնման (DFS) էքսպոնենցիալ ժամանակային բարդության սահմանափակումներից և հաշվարկել բազային սահմաններից դուրս գտնվող մեծ գրաֆների ցիկլերի քանակը՝ կիրառելով դինամիկ ծրագրավորում կամ անալիտիկ կանխատեսում: Մա ապահովում է հզոր մաթեմատիկական ապարատ, որտեղ ալգորիթմի արդյունքները ծառայում են ոչ միայն որպես վերջնական տվյալներ, այլև որպես մուտքային պարամետրեր՝ հաջորդող մակարդակների համար տեսական հաշվարկներ իրականացնելու նպատակով:

ԵԶՐԱԿԱՅՈՒԹՅՈՒՆ

Ավարտական աշխատանքի / մագիստրոսական թեզի շրջանակներում կատարված հետազոտական և ծրագրավորման աշխատանքների արդյունքում կարելի է անել հետևյալ հիմնական եզրահանգումները.

- 1. Տեսական և ալգորիթմական հիմքեր:** Ուսումնասիրվել են գրաֆների տեսության և Համիլտոնյան ցիկլերի հիմնարար հատկությունները: Մաթեմատիկորեն հիմնավորվել և ծրագրային մակարդակում որպես ֆիլտր են ներդրվել գրաֆների զույգությանը վերաբերող սահմանափակումները, ինչը բացառում է անիրագործելի տարբերակների անիմաստ հաշվարկումը:
- 2. Ճարտարապետական լուծումների էվոլյուցիա և օպտիմիզացիա:** Փորձարարական ճանապարհով ապացուցվել է, որ Համիլտոնյան ցիկլերի խիստ ճյուղավորված որոնման խնդիրներում GPU զուգահեռացումը (CUDA) և մաքուր ռեկուրսիվ մեթոդներն արդյունավետ չեն՝ հոսքերի տարամիտման և հիշողության գերբեռնվածության պատճառով: Որպես օպտիմալ այլընտրանք՝ նախագծվել է բիթային դիմակների (bitmasks) և JIT-կոմպիլյացիայի համակցությամբ աշխատող ալգորիթմ: Ալգորիթմական էտման (pruning) մեթոդիկան՝ դեռևս չայցելված գագաթների կապակցվածության մշտական ստուգման միջոցով, հնարավորություն է տվել կտրուկ նվազեցնել որոնման ծառի ծավալը:
- 3. Արդյունքների գնեքացիա և վիզուալիզացիա:** Ստեղծվել է ճշգրիտ հաշվարկման Բարձր կատարողականությամբ գործիքակազմ, որը հաջողությամբ գտել է 1,024,028 ցիկլ 10x6 չափանի գրաֆում՝ ընդամենը 2 րոպեի ընթացքում: Միաժամանակ, էքսպոնենցիալ աճի պատճառով անհասանելի մեծ չափերի գրաֆների համար նախագծվել է հիբրիդային էվրիստիկ ալգորիթմ՝ օժտված ադապտիվ վերագործարկման, վիզուալիզացիայի և ջերմային քարտեզագրման մոդուլներով:

Այսպիսով, հետազոտության առաջադրված նպատակներն ու խնդիրները
ամբողջությամբ իրագործվել են: Մշակված ծրագրային համալիրները և
օպտիմիզացիոն մոտեցումները կարող են ծառայել որպես հուսալի և արդյունավետ
հիմք՝ ցանցային գրաֆներում երթուղավորման ու կոմբինատորային օպտիմիզացիայի
խնդիրների հետագա գիտական ուսումնասիրությունների համար: